

Link state Routing Practice problems

ANUSHA N

A routing table can be either static or dynamic. A static table is one with manual entries. A dynamic table is one that is updated automatically when there is a change somewhere in the Internet. A routing protocol is a combination of rules and procedures that lets routers in the Internet inform each other of changes.

Topics discussed in this section:

Optimization

Intra- and Interdomain Routing

Distance Vector Routing and RIP

Link State Routing and OSPF

Path Vector Routing and BGP

Optimization

A router receives a packet from a network and passes it to another networks. when it receives a packet , to which network should it pass the packet? The decision is based on optimization: Which of the available pathways is the optimum pathway?

Intra domain and Inter domain Routing

An autonomous system is a group of networks and routers under the authority of a single administration.

Routing inside an autonomous system is referred to as **intradomain routing**

Routing between autonomous system is referred to as **interdomain routing**

Figure 22.12 *Autonomous systems*

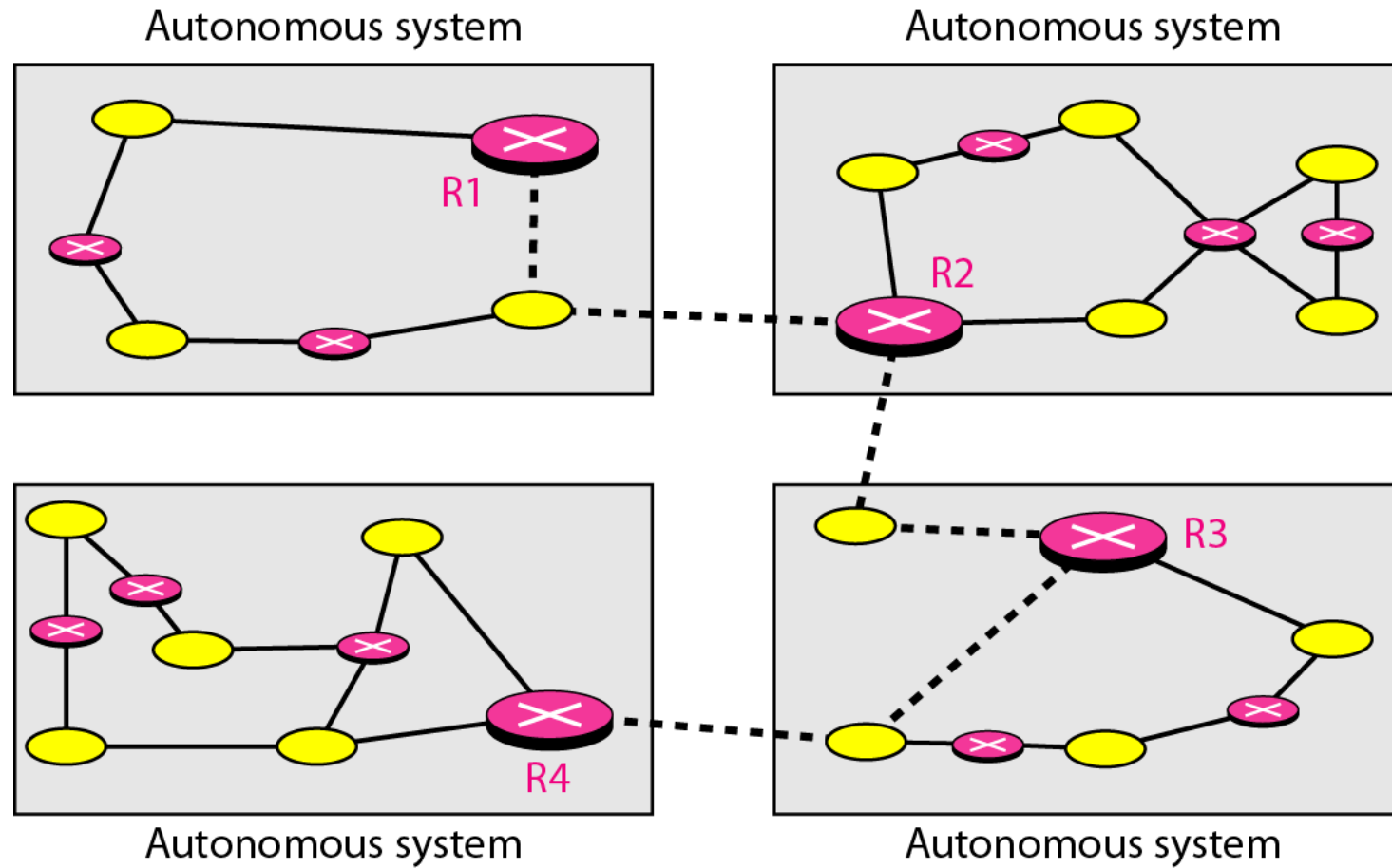
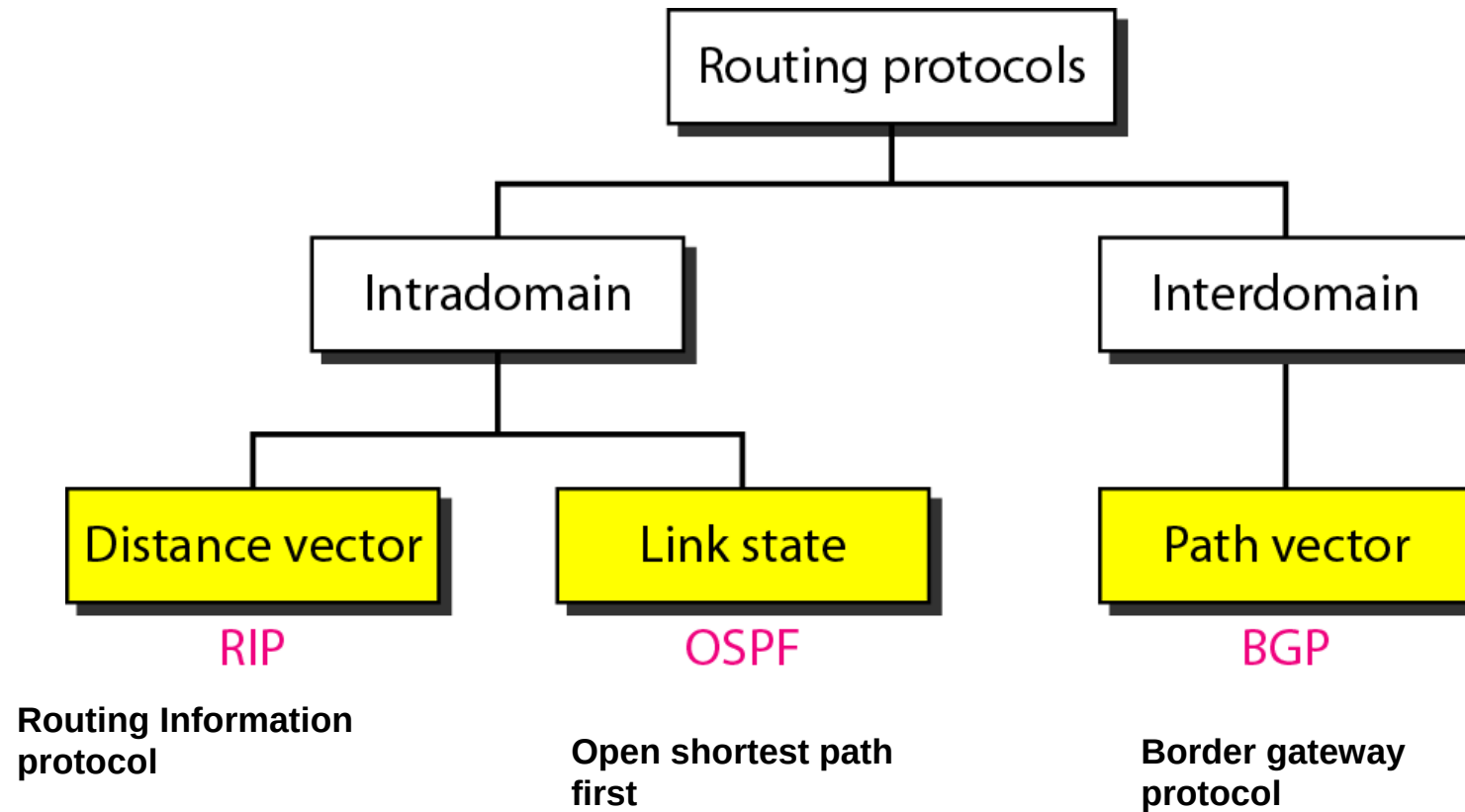
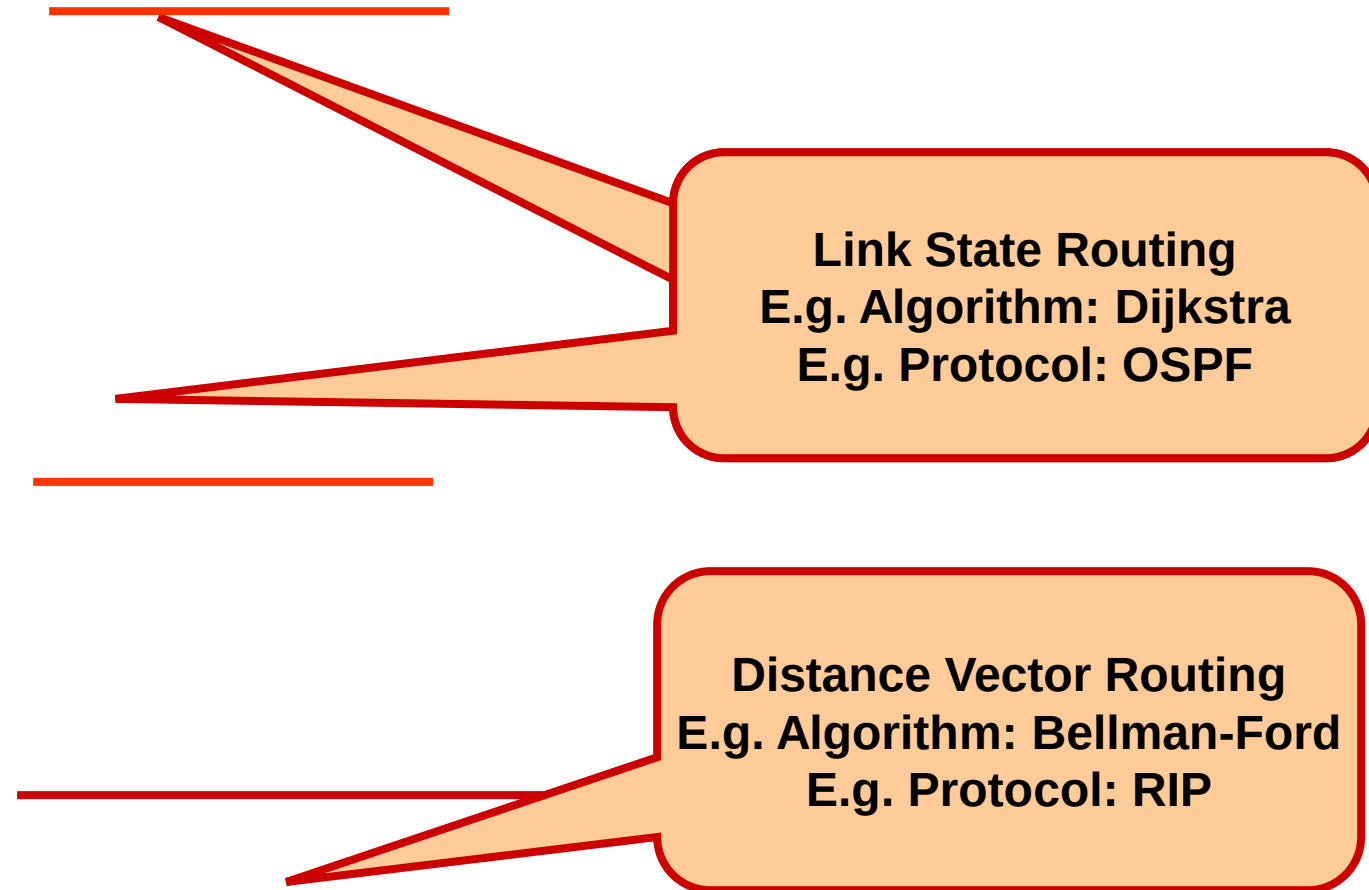


Figure 22.13 *Popular routing protocols*



Routing Requires Knowing Network



Link State Routing

- Link state routing is a technique in which each router shares the knowledge of its neighborhood with every other router in the internetwork.

The three keys to understand the Link State Routing algorithm:

- **Knowledge about the neighborhood:** Instead of sending its routing table, a router sends the information about its neighborhood only. A router broadcast its identities and cost of the directly attached links to other routers.
- **Flooding:** Each router sends the information to every other router on the internetwork. This process is known as Flooding. Every router that receives the packet sends the copies to all its neighbors. Finally, each and every router receives a copy of the same information.
- **Information sharing:** A router sends the information to every other router only when the change occurs in the information.

Link State Routing

Link State Routing has two phases:

Reliable Flooding

- Initial state: Each node knows the cost of its neighbors.
- Final state: Each node knows the entire graph.

Route Calculation

- Each node uses Dijkstra's algorithm on the graph to calculate the optimal routes to all nodes.
- The Link state routing algorithm is also known as Dijkstra's algorithm which is used to find the shortest path from one node to every other node in the network.
- The Dijkstra's algorithm is an iterative, and it has the property that after kth iteration of the algorithm, the least cost paths are well known for k destination nodes.

Figure 22.20 *Concept of link state routing*

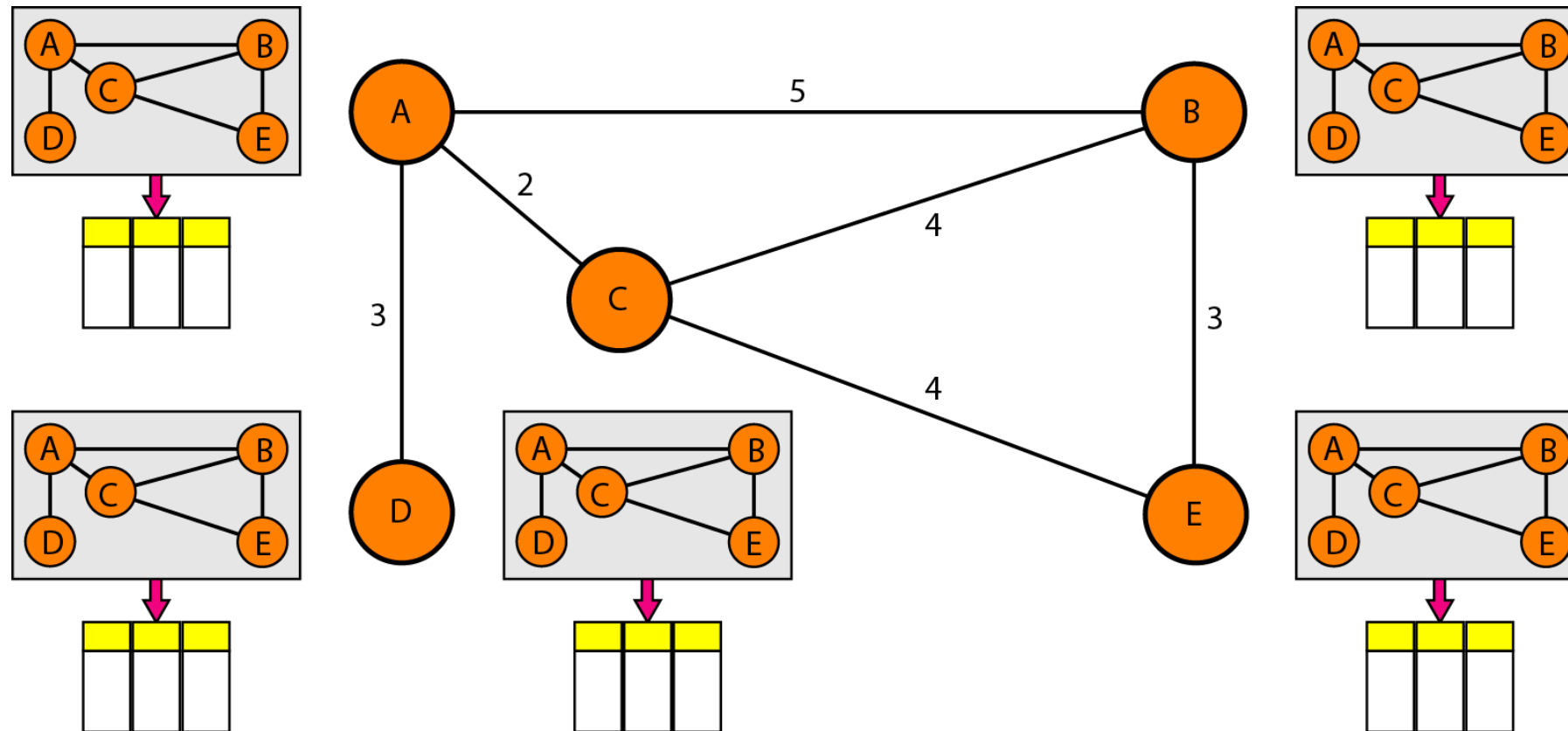


Figure 22.21 *Link state knowledge*

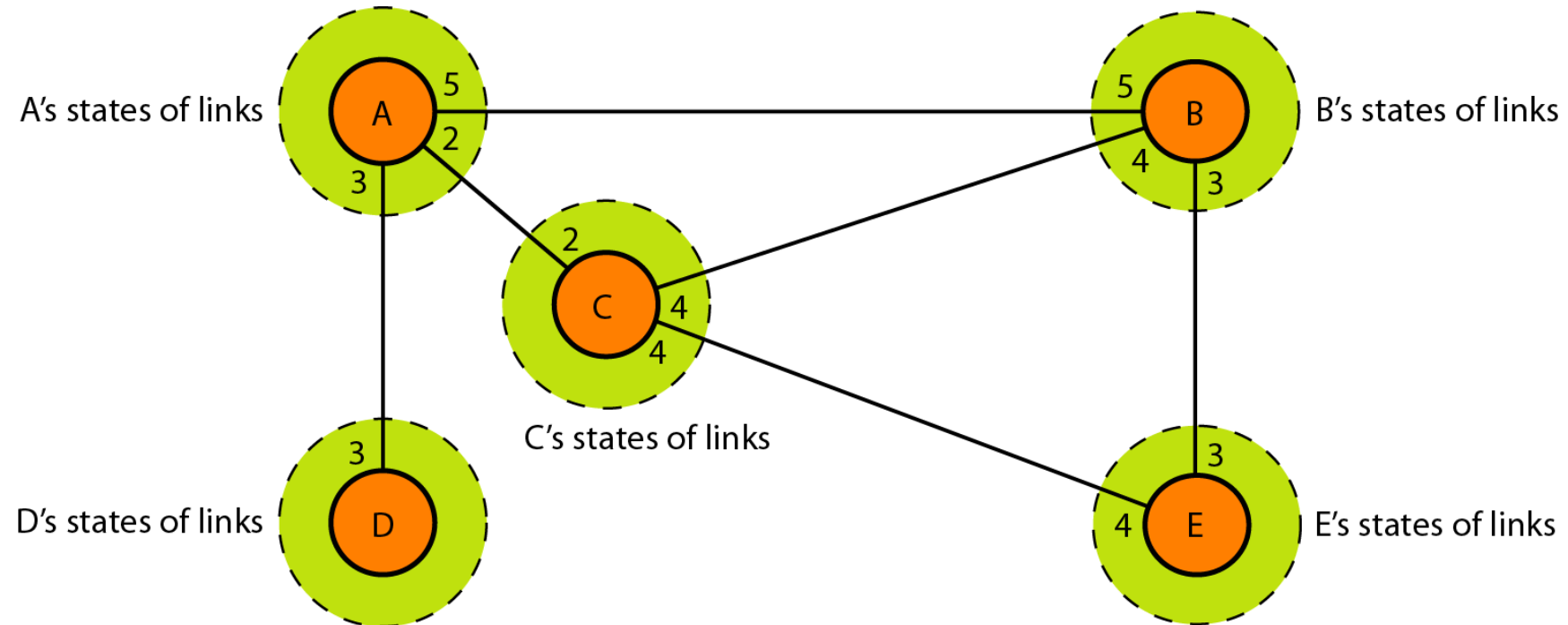


Figure 22.22 *Dijkstra algorithm*

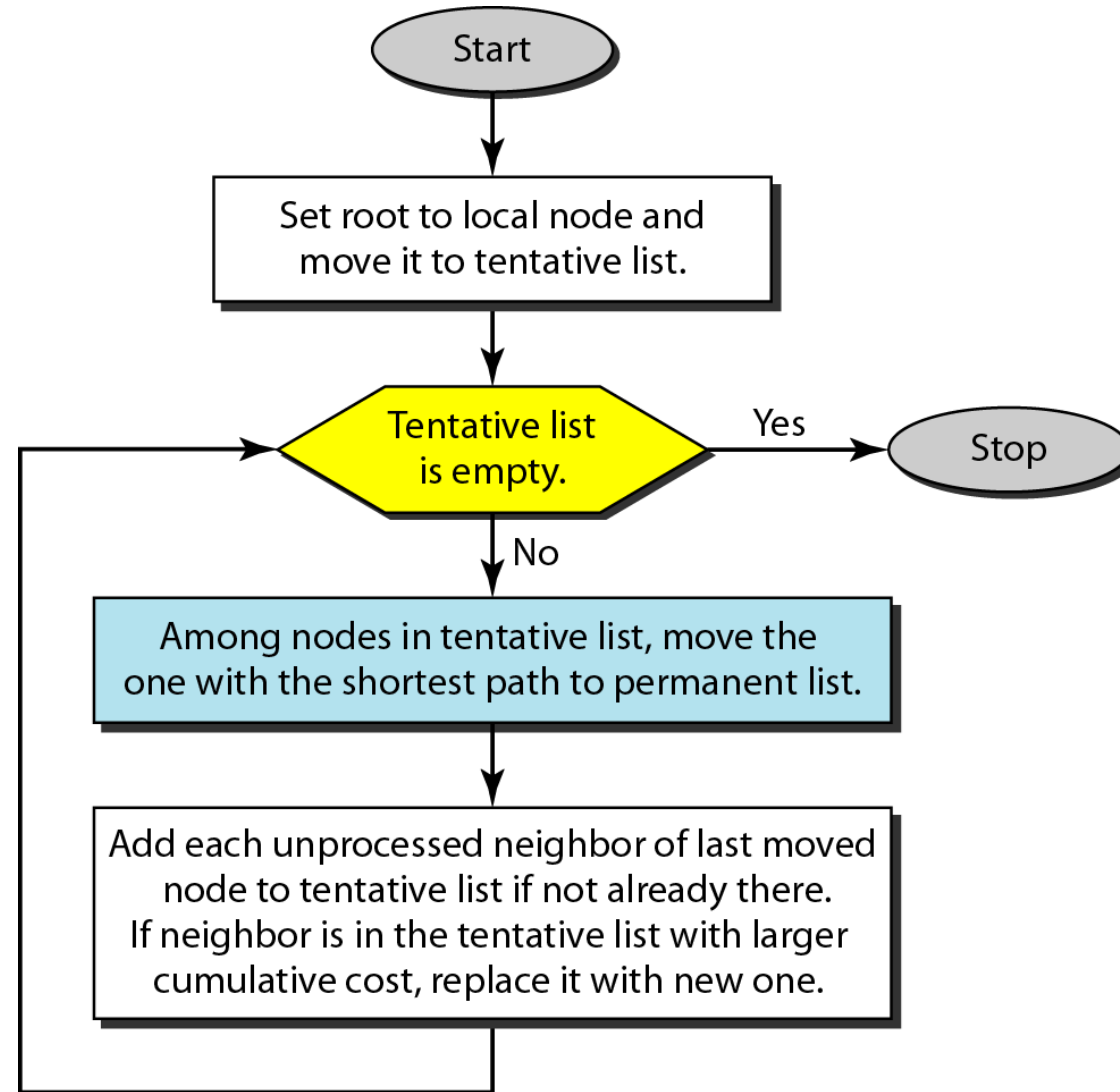


Figure 22.23 *Example of formation of shortest path tree*

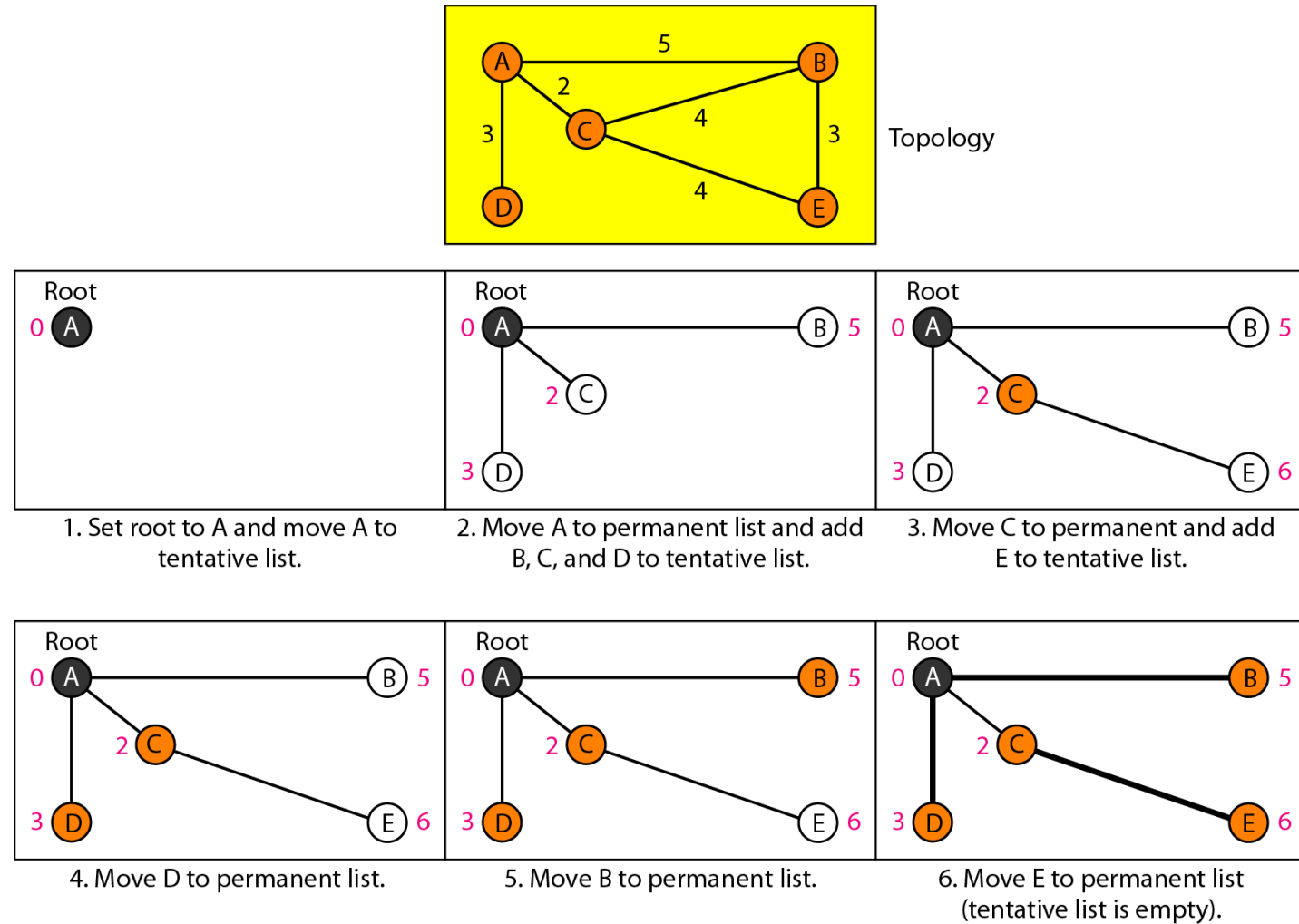
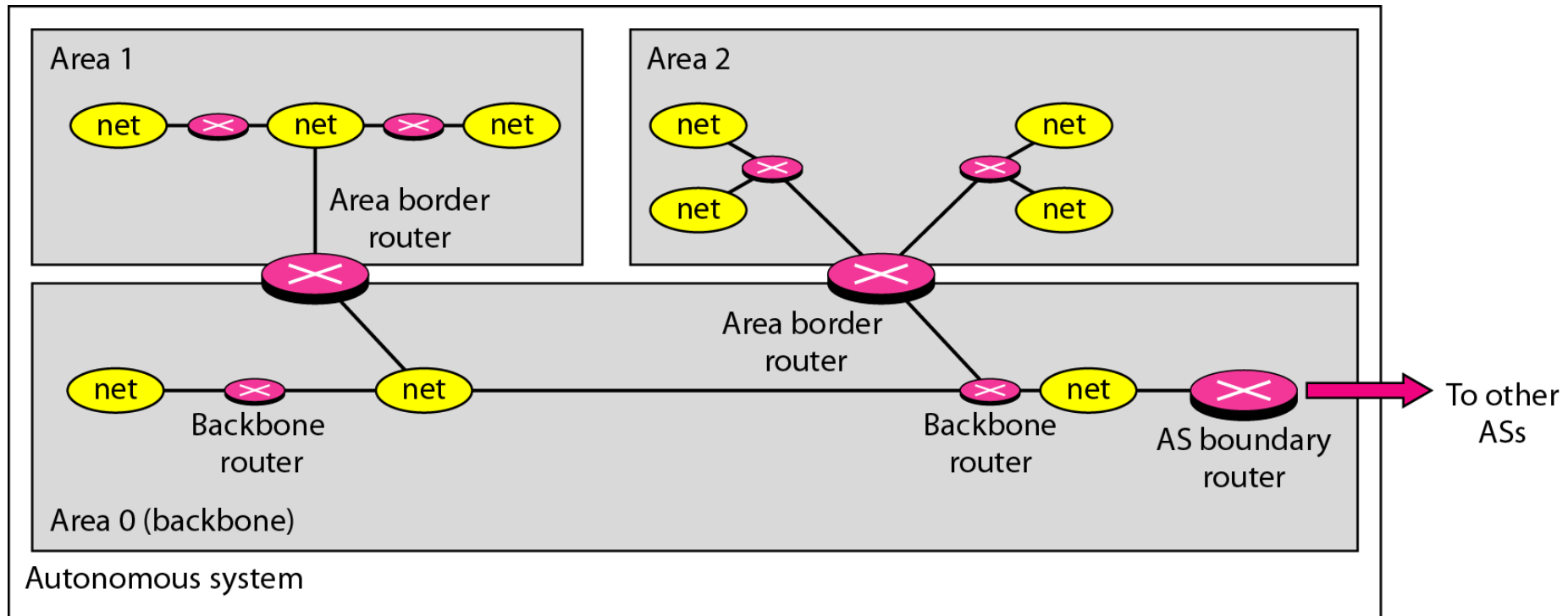


Table 22.2 *Routing table for node A*

<i>Node</i>	<i>Cost</i>	<i>Next Router</i>
A	0	—
B	5	—
C	2	—
D	3	—
E	6	C

Figure 22.24 *Areas in an autonomous system*



A Link State Routing Algorithm

Dijkstra's algorithm

- net topology, link costs known to all nodes
 - accomplished via "link state broadcast"
 - all nodes have same info
- computes least cost paths from one node ("source") to all other nodes
 - gives routing table for that node
- iterative: after k iterations, know least cost path to k dest.'s

Notation:

- $c(i,j)$: link cost from node i to j . cost infinite if not direct neighbors
- $D(v)$: current value of cost of path from source to dest. v
- $p(v)$: predecessor node along path from source to v , that is next v
- N : set of nodes whose least cost path definitively known

Dijkstra's Algorithm

1 **Initialization:**

2 $N = \{A\}$

3 for all nodes v

4 if v adjacent to A

5 then $D(v) = c(A, v)$

6 else $D(v) = \text{infty}$

7

8 **Loop**

9 find w not in N such that $D(w)$ is a minimum

10 add w to N

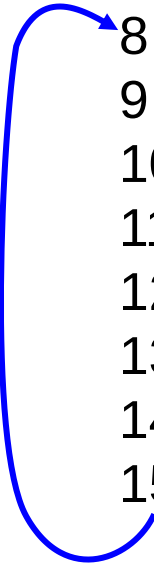
11 update $D(v)$ for all v adjacent to w and not in N :

12 $D(v) = \min(D(v), D(w) + c(w, v))$

13 /* new cost to v is either old cost to v or known

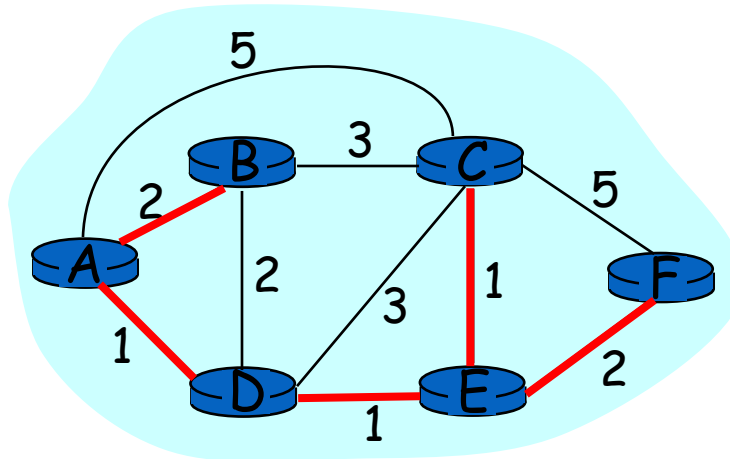
14 shortest path cost to w plus cost from w to v */

15 **until all nodes in N**



Dijkstra's Algorithm: An Example

Step	start N	D(B),p(B)	D(C),p(C)	D(D),p(D)	D(E),p(E)	D(F),p(F)
→ 0	A	2,A	5,A	1,A	infinity	infinity
→ 1	AD	2,A	4,D		2,D	infinity
→ 2	ADE	2,A	3,E			4,E
→ 3	ADEB		3,E			4,E
→ 4	ADEBC					4,E
5	ADEBCF					



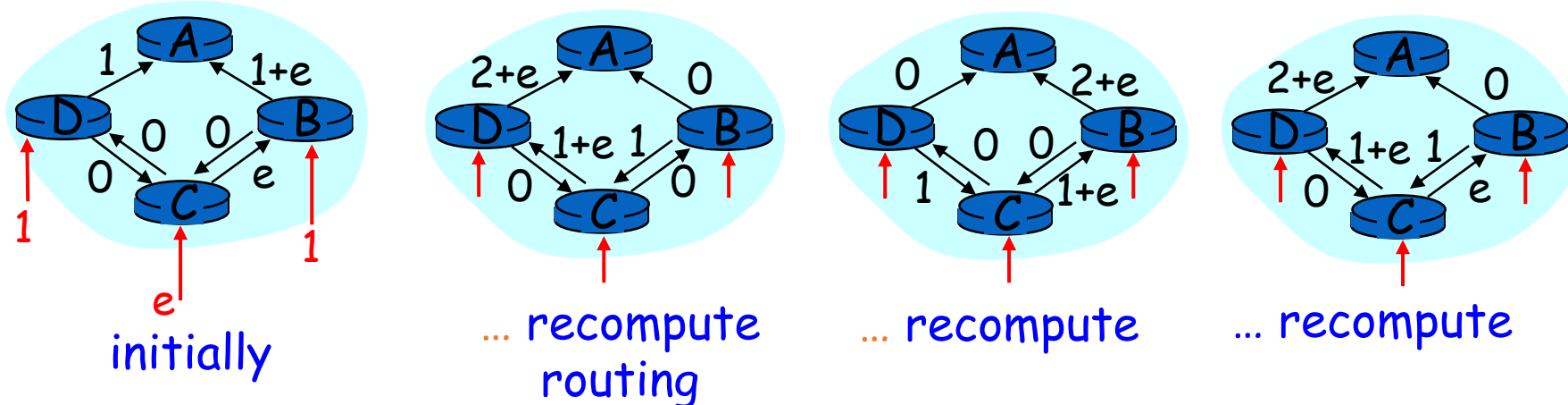
Dijkstra's Algorithm: Discussion

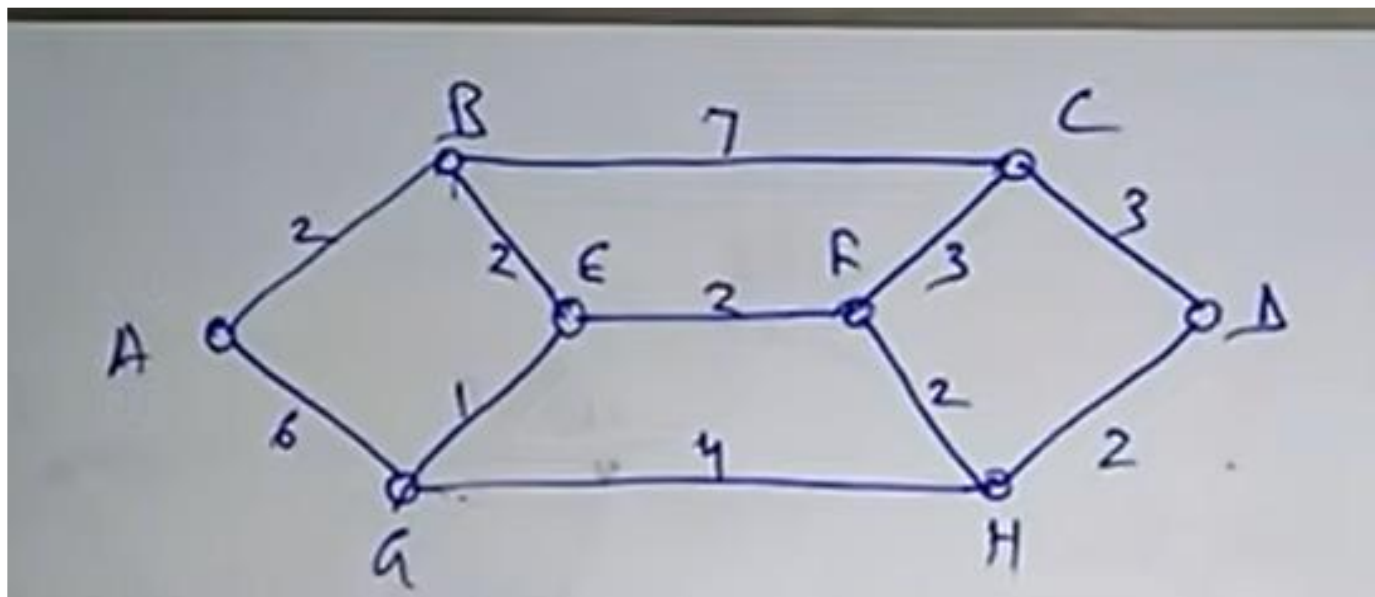
Algorithm complexity: n nodes

- each iteration: need to check all nodes, w , not in N
- $n*(n+1)/2$ comparisons: $O(n^2)$
- more efficient implementations possible: $O(n \log n)$

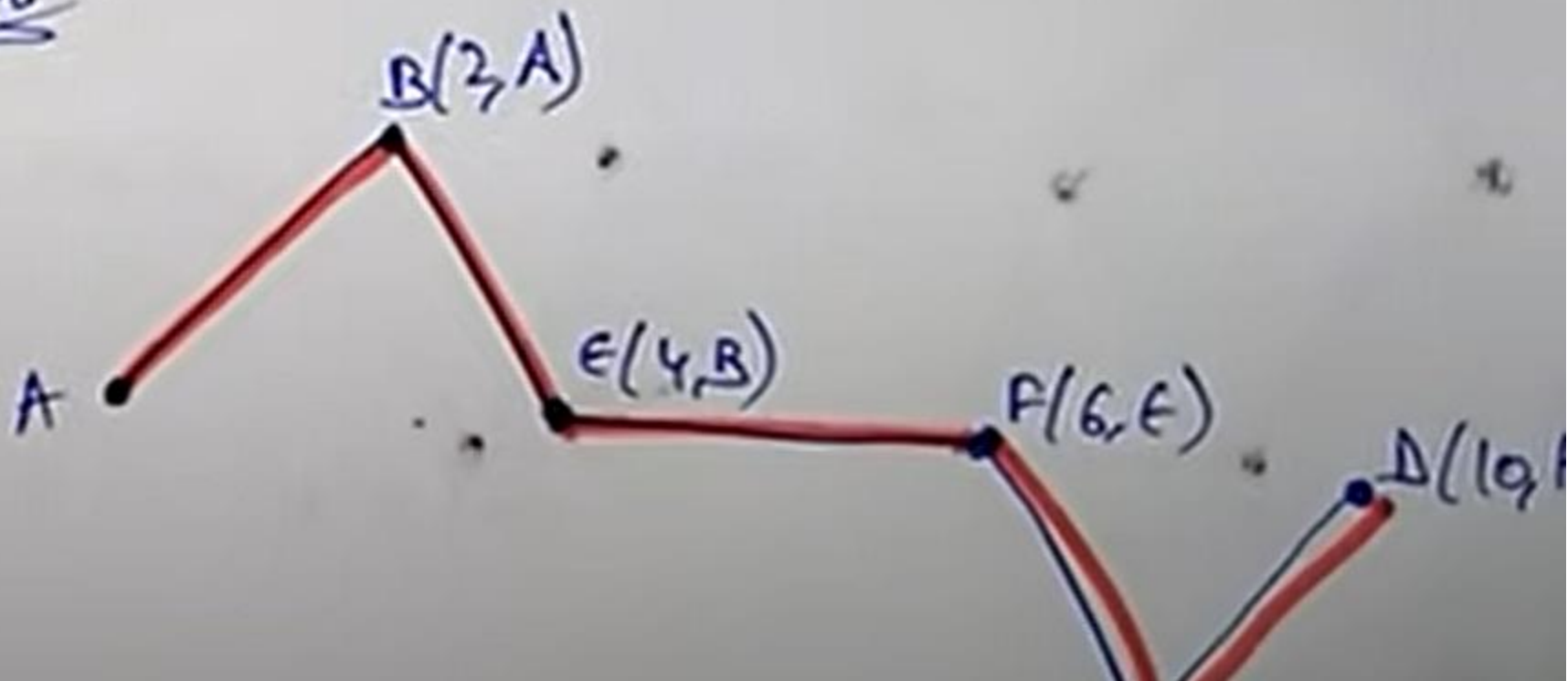
Oscillations possible:

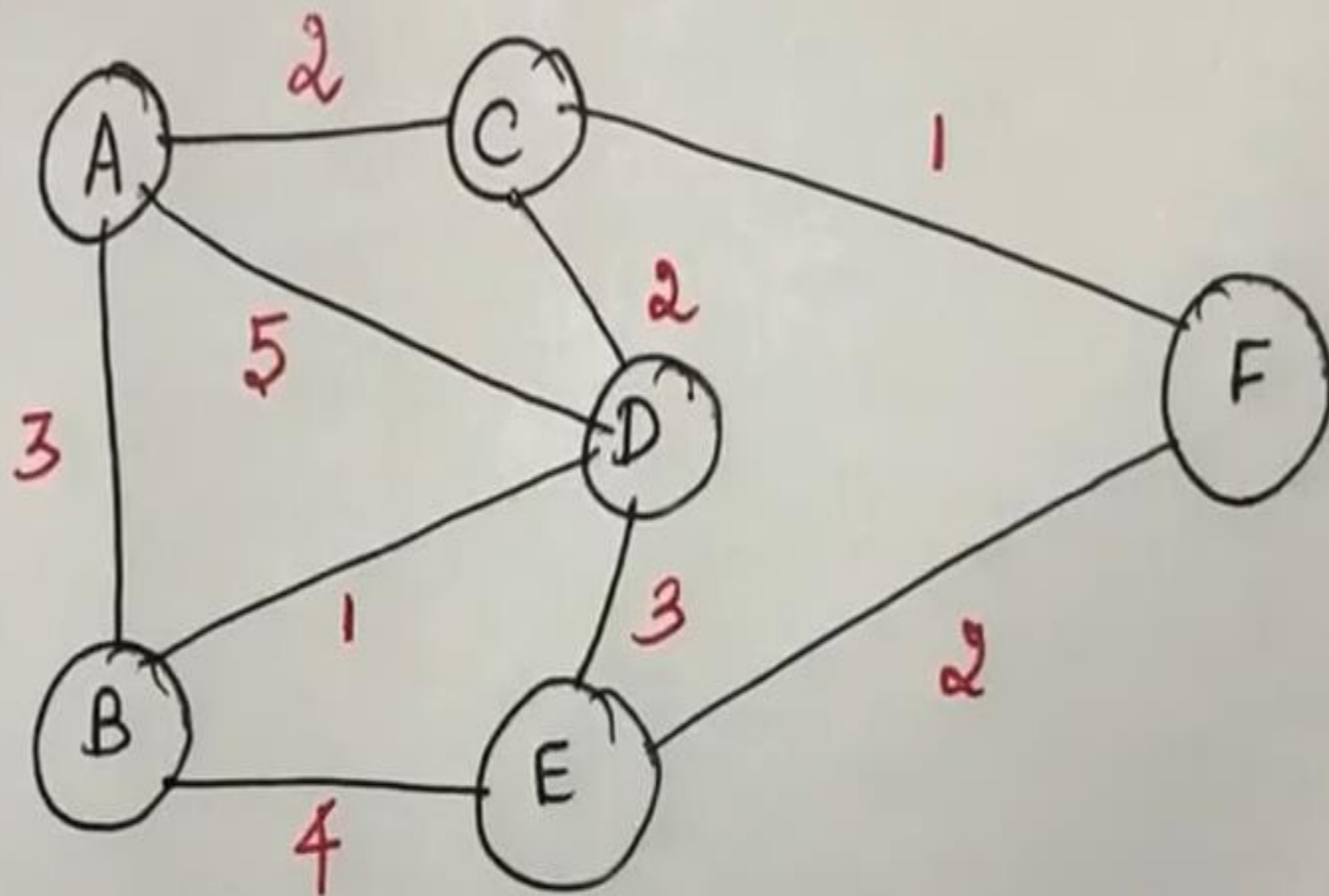
- e.g., link cost = amount of carried traffic

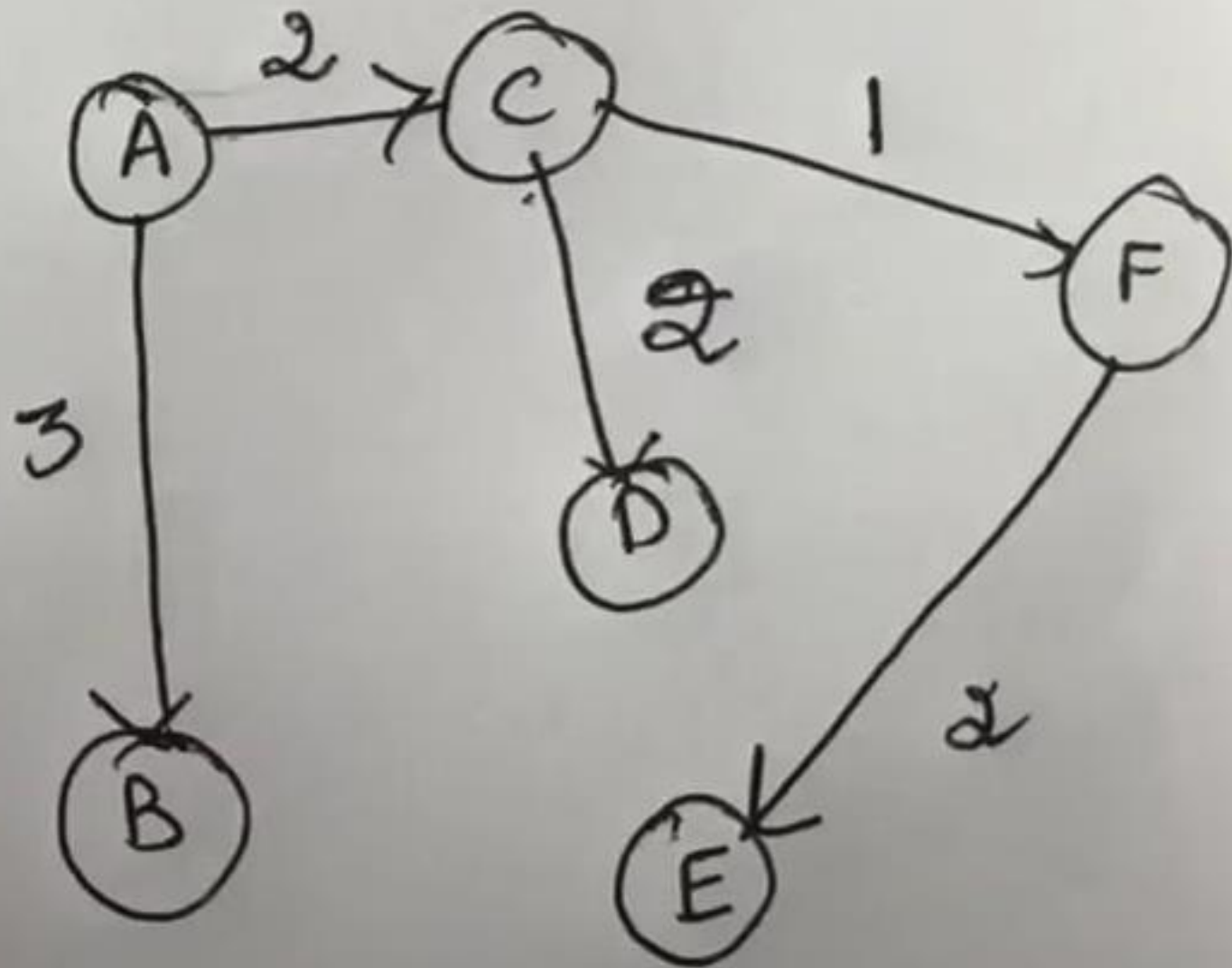




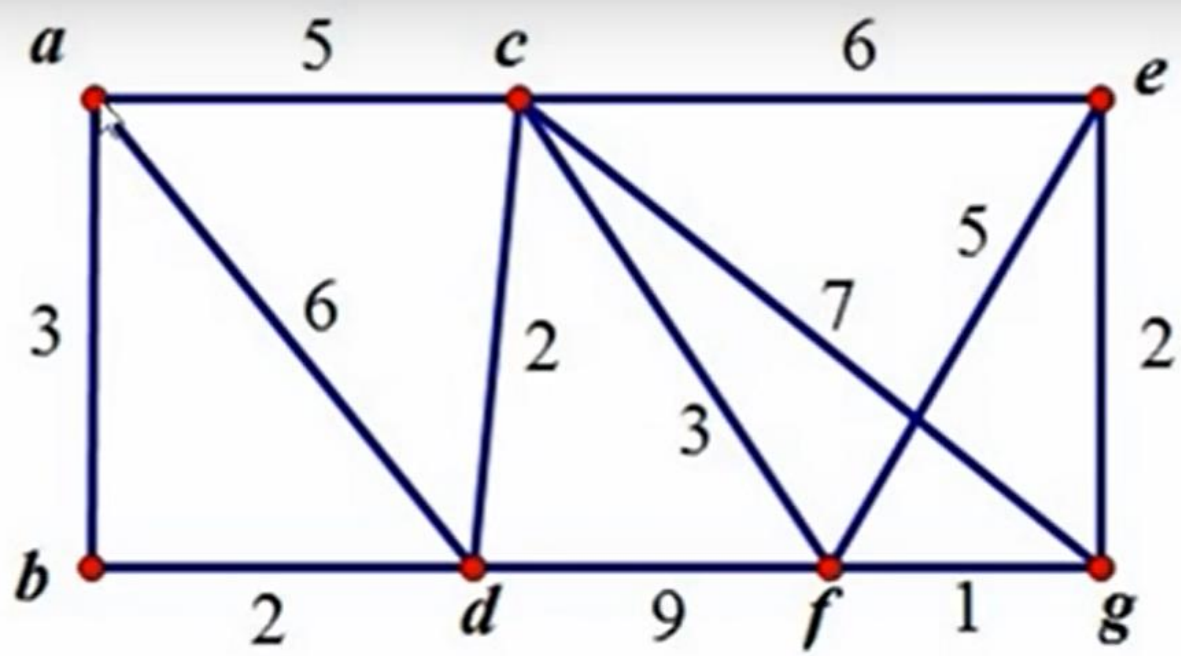
Step 16!



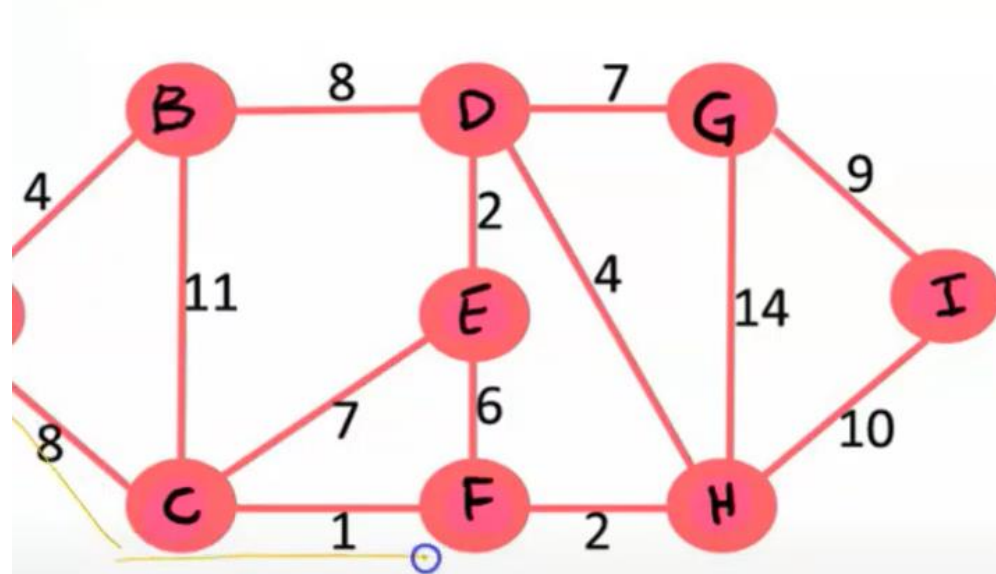




Iteration	Tree	B	C	D	E	F
Initial	$\{A\}$	3	(2)	5	∞	∞
1	$\{A, C\}$	(3)	-	4	∞	3
2	$\{A, B, C\}$	-	-	4	7	(3)
3	$\{A, B, C, F\}$	-	-	(4)	5	-
4	$\{A, B, C, D, F\}$	-	-	-	5	-
	$\{A, B, C, D, E, F\}$	-	-	-	-	-



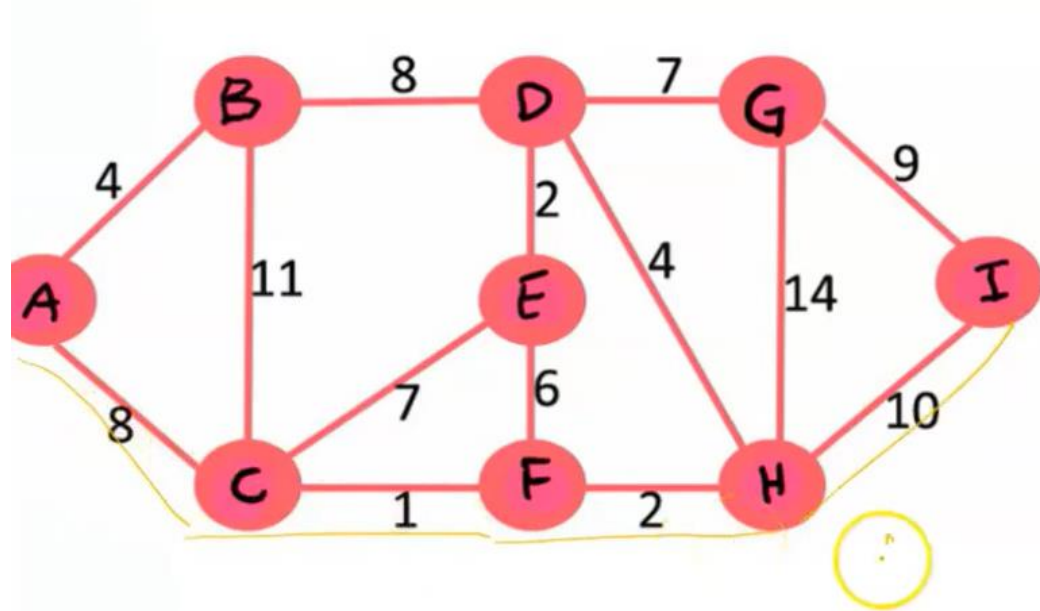
v	a	b	c	d	e	f	g
a	0_a	3_a	5_a	6_a	∞_a	∞_a	∞_a
b	0_a	3_a	5_a	5_b	∞_a	∞_a	∞_a
c	0_a	3_a	5_a	5_b	11_c	8_c	12_c
d	0_a	3_a	5_a	5_b	11_c	8_c	12_c
f	0_a	3_a	5_a	5_b	11_c	8_c	9_f



Find the shortest path from A to I

$$ACFHI = 21$$

	B	C	D	E	F	G	H	I
A	4	8	x	x	x	x	x	x
B	4	$11+4=15$ 8	$8+4=12$ 12	x	x	x	x	x
C	4	8	12	$7+8=15$ 15	$1+8=9$ 9	x	x	x
F	4	8	12	$6+9=15$ 15	9	x	$2+9=11$ 11	x
H	4	8	$4+11=15$ 12	15	9	$14+11=25$ 25	11	$10+11=21$ 21
D	4	8	12	$2+12=14$ 14	9	$7+12=19$ 19	11	21
E	4	8	12	14	9	19	11	21
G	4	8	12	14	9	19	11	$9+19=28$ 21
I	4	8	12	14	9	19	11	21



Find the shortest path from A to I

ACFHI = 21

	B	C	D	E	F	G	H	I
A	4	8	x	x	x	x	x	x
B	4	8 ¹¹⁺⁴⁼¹⁵	12 ⁸⁺⁴⁼¹²	x	x	x	x	x
C	/	8	12	15 ⁷⁺⁸⁼¹⁵	9 ¹⁺⁸⁼⁹	x	x	x
F	/	/	12	15 ⁶⁺⁹⁼¹⁵	9	x	11 ²⁺⁹⁼¹¹	x
H	/	/	12 ⁴⁺¹¹⁼¹⁵	15	/	25 ¹⁴⁺¹¹⁼²⁵	11	21 ¹⁰⁺¹¹⁼²¹
D	/	/	12	14 ²⁺¹²⁼¹⁴	/	19 ⁷⁺¹²⁼¹⁹	/	21
E	/	/	/	14	/	19	/	21
G	/	/	/	/	/	19	/	21 ⁹⁺¹⁹⁼²⁸
I	/	/	/	/	/	/	/	21